

# **SIMATS ENGINEERING**

## **ASSESSMENT TOOL 1**

**COURSE CODE: ITA0605**

**COURSE NAME: MACHINE LEARNING**

### **COURSE OUTCOME COVERED**

**CO1:** *Learning Problems, Perspectives and Issues, Concept Learning, Version Spaces & Candidate Elimination, Inductive Bias, Decision Tree Learning, Representation & Heuristic Search (BL1, BL2)*

*Assessment Tool Weightage: 40%*

**QUESTIONS: 15**

**Total Marks: 25**

Annexure A

### **APPLICATION BASED QUESTIONS**

#### ***Code of Conduct:***

*I Rishwanth. L / 192424100 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

***Signature of the student:***

## Question

11. In Shop X, there are 20 bottles of pure oil and 30 bottles of adulterated oil, while in Shop Y, there are 40 bottles of pure oil and 20 bottles of adulterated oil. A bottle is selected at random from one of the shops, with Shop Y being twice as likely to be chosen as Shop X. The selected bottle is found to be adulterated. (5 Marks)

## Python Code

```
P_X = 1 / 3
P_Y = 2 / 3
P_A_given_X = 30 / 50
P_A_given_Y = 20 / 60
P_A = (P_A_given_X * P_X) + (P_A_given_Y * P_Y)
P_X_given_A = (P_A_given_X * P_X) / P_A
P_Y_given_A = (P_A_given_Y * P_Y) / P_A

print("==== BAYES THEOREM =====")
print(f"Probability of Shop X being selected      : {P_X:.4f}")
print(f"Probability of Shop Y being selected      : {P_Y:.4f}\n")
print(f"P(Adulterated | Shop X) = {P_A_given_X:.4f}")
print(f"P(Adulterated | Shop Y) = {P_A_given_Y:.4f}\n")
print(f"Total Probability of Adulterated Bottle = {P_A:.4f}\n")
print("Posterior Probabilities:")
print(f"P(Shop X | Adulterated) = {P_X_given_A:.4f}
({P_X_given_A*100:.2f}%)")
print(f"P(Shop Y | Adulterated) = {P_Y_given_A:.4f}
({P_Y_given_A*100:.2f}%)")
```

## Output

```
==== BAYES THEOREM =====
Probability of Shop X being selected      : 0.3333
Probability of Shop Y being selected      : 0.6667

P(Adulterated | Shop X) = 0.6000
P(Adulterated | Shop Y) = 0.3333

Total Probability of Adulterated Bottle = 0.4222

Posterior Probabilities:
P(Shop X | Adulterated) = 0.4737 (47.37%)
P(Shop Y | Adulterated) = 0.5263 (52.63%)
```

## Question

12. A warehouse has two sections. Section A contains 15 defective items and 35 non-defective items, while Section B contains 25 defective items and 25 non-defective items. One section is chosen at random, but Section A is three times as likely to be chosen as Section B. An item selected from the chosen section is found to be defective. (5 Marks)

## Python Code

```
P_A = 3 / 4
P_B = 1 / 4
P_D_given_A = 15 / 50
P_D_given_B = 25 / 50
P_D = (P_D_given_A * P_A) + (P_D_given_B * P_B)
P_A_given_D = (P_D_given_A * P_A) / P_D
P_B_given_D = (P_D_given_B * P_B) / P_D
print("==== BAYES THEOREM ====")
print(f"Probability of choosing Section A : {P_A:.4f}")
print(f"Probability of choosing Section B : {P_B:.4f}\n")
print(f"P(Defective | Section A) = {P_D_given_A:.4f}")
print(f"P(Defective | Section B) = {P_D_given_B:.4f}\n")
print(f"Total Probability of Defective Item = {P_D:.4f}\n")
print("Posterior Probabilities:")
print(f"P(Section A | Defective) = {P_A_given_D:.4f} ({P_A_given_D*100:.2f}%)")
print(f"P(Section B | Defective) = {P_B_given_D:.4f} ({P_B_given_D*100:.2f}%)")
```

## Output

```
==== BAYES THEOREM ====
Probability of choosing Section A : 0.7500
Probability of choosing Section B : 0.2500

P(Defective | Section A) = 0.3000
P(Defective | Section B) = 0.5000

Total Probability of Defective Item = 0.3500

Posterior Probabilities:
P(Section A | Defective) = 0.6429 (64.29%)
P(Section B | Defective) = 0.3571 (35.71%)
```

## Question

13. In a binary classification problem using a deep learning model with a K-Nearest Neighbor classifier, the model produces True Positive (TP)=120 and True Negative (TN)=150. The dataset contains 1000 samples per class, and 20% of the data is used for testing. Find Accuracy, Precision, Sensitivity (Recall), Specificity. (5 Marks)

## Python Code

```
TP = 120
TN = 150
samples_per_class = 1000
test_ratio = 0.20
positive_test = int(samples_per_class * test_ratio)
negative_test = int(samples_per_class * test_ratio)
FN = positive_test - TP
FP = negative_test - TN
accuracy = (TP + TN) / (positive_test + negative_test)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)
print("==== CLASSIFICATION REPORT ====")
print(f"Positive Test Samples : {positive_test}")
print(f"Negative Test Samples : {negative_test}\n")
print("Confusion Matrix")
print("-----")
print(f"TP = {TP}")
print(f"FP = {FP}")
print(f"FN = {FN}")
print(f"TN = {TN}\n")
print(f"Accuracy   : {accuracy*100:.2f}%")
print(f"Precision   : {precision*100:.2f}%")
print(f"Recall      : {recall*100:.2f}%")
print(f"Specificity: {specificity*100:.2f}%")
```

## Output

```
==== CLASSIFICATION REPORT ====
Positive Test Samples : 200
Negative Test Samples : 200

Confusion Matrix
-----
TP = 120
FP = 50
FN = 80
TN = 150

Accuracy   : 67.50%
Precision   : 70.59%
Recall      : 60.00%
Specificity: 75.00%
```

## Question

14. A medical image classification system using a deep learning framework combined with K-Nearest Neighbor (KNN) yields TP=180 and TN=160. Each class contains 1200 samples, and 25% of the dataset is used as test data. Calculate Accuracy, Precision, Sensitivity, Specificity. (5 Marks)

## Python Code

```
TP = 180
TN = 160
samples_per_class = 1200
test_ratio = 0.25
positive_test = int(samples_per_class * test_ratio)
negative_test = int(samples_per_class * test_ratio)
FN = positive_test - TP
FP = negative_test - TN
accuracy = (TP + TN) / (positive_test + negative_test)
precision = TP / (TP + FP)
recall = TP / (TP + FN)
specificity = TN / (TN + FP)

print("==== CLASSIFICATION REPORT ====")
print(f"Positive Test Samples : {positive_test}")
print(f"Negative Test Samples : {negative_test}\n")
print("Confusion Matrix")
print("-----")
print(f"TP = {TP}")
print(f"FP = {FP}")
print(f"FN = {FN}")
print(f"TN = {TN}\n")
print(f"Accuracy   : {accuracy*100:.2f}%")
print(f"Precision   : {precision*100:.2f}%")
print(f"Recall       : {recall*100:.2f}%")
print(f"Specificity: {specificity*100:.2f}%")
```

## Output

```
==== CLASSIFICATION REPORT ====
Positive Test Samples : 300
Negative Test Samples : 300

Confusion Matrix
-----
TP = 180
FP = 140
FN = 120
TN = 160

Accuracy   : 56.67%
Precision   : 56.25%
Recall      : 60.00%
Specificity: 53.33%
```

## Question

15. A deep learning model is trained for a classification task, and the following loss values are observed... Answer the questions.

## Python Code

```
import matplotlib.pyplot as plt
epochs = [1,2,3,4,5,6,7,8]
training_loss = [2.8,2.4,2.1,1.9,1.7,1.5,1.3,1.2]
validation_loss = [2.6,2.2,1.9,1.8,1.9,2.1,2.4,2.7]
best_epoch = validation_loss.index(min(validation_loss)) + 1
print("==== LOSS ANALYSIS =====")
print(f"Best Generalization Epoch : {best_epoch}")
if best_epoch < len(epochs):
    print(f"Overfitting Starts After : Epoch {best_epoch}")
print("\nSuggestions to Reduce Overfitting")
print("-----")
print("1. Early Stopping")
print("2. Dropout Regularization")
plt.figure(figsize=(8,5))
plt.plot(epochs, training_loss, marker='o', label="Training Loss")
plt.plot(epochs, validation_loss, marker='s', label="Validation Loss")
plt.title("Training Loss vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.xticks(epochs)
plt.grid(True)
plt.legend()
plt.show()
```

## Output

